Práctica de laboratorio: Cifrar y descifrar datos con OpenSSL utilizando criptografía simétrica

Objetivos

Parte 1: Cifrar mensajes con OpenSSL utilizando simétrica

Parte 2: Descifrar mensajes con OpenSSL utilizando simétrica

Antecedentes / Escenario

OpenSSL[1] es un proyecto de código abierto que proporciona un kit de herramientas robusto, comercial y completo para los protocolos de Seguridad de capa de transporte Transport Layer Security (TLS) y Capa de sockets segura, Secure Sockets Layer (SSL). También es una biblioteca de criptografía de uso general.

Nota: Si bien OpenSSL es la biblioteca de criptografía obligada, el uso que se presenta en esta práctica de laboratorio NO se recomienda para lograr una protección sólida. A continuación se presentan dos problemas de seguridad con esta práctica:

1) El método que se describe en esta práctica utiliza un función de derivación de claves débiles. La ÚNICA medida de seguridad es una contraseña muy fuerte.

2) El método que se describe en esta práctica no garantiza la integridad del archivo de texto.

Esta práctica de laboratorio debe utilizarse solo con fines instructivos. Los métodos aquí presentado NO se deben emplear para asegurar datos realmente sensibles.

Recursos necesarios

- Máquina virtual CyberOps Workstation
- Acceso a Internet

[1] https://www.openssl.org/

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

Práctica de laboratorio: Cifrar y descifrar datos con OpenSSL

Parte 1: Cifrar mensajes con OpenSSL utilizando criptografía simétrica

OpenSSL se puede utilizar como una herramienta independiente para el cifrado. Aunque pueden utilizarse muchos algoritmos de cifrado, esta práctica de laboratorio se enfoca en AES. Si quieren utilizar AES para cifrar un archivo de texto directamente desde la línea de comando utilizando OpenSSL, sigan los pasos que se indican a continuación:

Paso 1: Cifrar un archivo de texto

a. Iniciar sesión en la VM CyberOPS Workstation y abrir una ventana del Terminal.

b. El archivo de texto que se debe cifrar se encuentra en el directorio /home/analyst/lab.support.files/, pasen a ese directorio:

```
[analyst@secOps ~]$ cd ./lab.support.files/
[analyst@secOps lab.support.files]$
```

c. Escriba el siguiente comando a continuación para generar un archivo de texto plano:

```
[analyst@secOps lab.support.files]$ cat letter_to_grandma.txt

Hola abuela,

Te escribo esta carta para agradecerte el pan casero con
chicharrones y el salame de Colonia Caroya que me enviaste.
¡Los probé esta mañana en el desayuno y son absolutamente
deliciosos!

Te deseo lo mejor.
Tu nieto feliz.

[analyst@secOps lab.support.files]$
```

d. En la misma ventana del terminal, escriba el siguiente comando para cifrar el archivo de texto. Utilizará AES-256 para cifrar el archivo de texto y guardar la versión cifrada como **message.enc**. OpenSSL le pedirá una contraseña y que la confirmen. Proporcione la contraseña tal como se les solicita y documentelá.

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc -in
letter_to_grandma.txt -out message.enc
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
[analyst@secOps lab.support.files]$
```

e. Cuando haya terminado el proceso, vuelvan a utilizar el comando **cat** para mostrar el contenido del archivo **message.enc**. Documentar esta experiencia.

```
[analyst@secOps lab.support.files]$ cat message.enc
```

f. Para que el archivo sea legible, vuelva a ejecutar el comando OpenSSL pero esta vez agreguen la opción **-a**. La opción **-a** le indica a OpenSSL que debe cifrar el mensaje cifrado con un método de codificación diferente a **Base64** antes de guardar el resultado en un archivo.

Nota: Base64 es un grupo de esquemas similares de codificación binario a texto que se utiliza para representar datos binarios en un formato de cadena ASCII.

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc -a -in
letter_to_grandma.txt -out message.enc
enter aes-256-cbc encryption password:
Verifying - enter aes-256-cbc encryption password:
```

g. Nuevamente, utilicen el comando **cat** para mostrar el contenido del archivo **message.enc**, que se acaba de generar otra vez. Documente esta experiencia y comparar con la experiencia del punto e). ¿Puede pensar en algún beneficio de tener **message.enc** codificado como Base64 ?

Parte 2: Descifrar mensajes con OpenSSL utilizando criptografía simétrica

Con un comando OpenSSL similar se puede descifrar **message.enc**.

a. Utilicen el siguiente comando para descifrar message.enc:

```
[analyst@secOps lab.support.files]$ openssl aes-256-cbc –a -d -in
message.enc -out decrypted_letter.txt
```

b. OpenSSL les pedirá la contraseña que se utilizó para cifrar el archivo. Vuelvan a introducir la misma contraseña.

c. Cuando OpenSSL termine de descifrar el archivo **message.enc**, lo guardará en un archivo de texto de nombre **decrypted_letter.txt**. Utilizar el comando **cat** para mostrar el contenido de **decrypted_letter.txt** Verificar que la carta se desencriptó correctamente. Observer que el comando que se utilizó para descifrar también contiene una opción **-a**. ¿ Puede explicarlo ?

```
[analyst@secOps lab.support.files]$ cat decrypted_letter.txt
```

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

Práctica de laboratorio: Cifrar y descifrar datos con OpenSSL utilizando criptografía asimétrica

Objetivos

Parte 1: Cifrar mensajes con OpenSSL utilizando asimétrica

Parte 2: Descifrar mensajes con OpenSSL utilizando asimétrica

Antecedentes / Escenario

La Criptografía de clave pública utiliza un par de claves por usuario o entidad para brindar servicios de seguridad. Una es la clave pública y se puede compartir libremente. La otra clave es privada y se supone que es secreta y nunca se comparte.

Para explorar el cifrado y descifrado de archivos, imagine a dos usuarios, **Alice** y **Bob**, que desean comunicarse entre sí intercambiando archivos cifrados mediante OpenSSL.

Las claves públicas se utilizan para brindar servicios de confidencialidad. Si **Alice** quiere enviar información confidencial a **Bob**, primero le pide a **Bob** su clave pública, luego **Alice** puede utilizarla para cifrar mensajes o archivos antes de enviárselos a **Bob**. La clave privada la utiliza **Bob** para el descifrado. La única forma en que puede descifrar el mensaje cifrado enviado por **Alice** es utilizando su clave privada. De ahí el descriptor "par de claves"; el juego de claves están relacionadas desde su creación.

Parte 1. Cifrar mensajes con OpenSSL utilizando criptografía asimétrica a)

Generar el par de claves

El primer paso es generar un par de claves. También necesitará una frase de contraseña, que debe utilizar siempre que use OpenSSL, así que asegúrese de recordarla. **Alice** genera su conjunto de pares de claves con el siguiente comando:

[analyst@secOps lab.support.files]$ **openssl genrsa -aes128 -out alice_private.pem 1024**

Este comando utiliza el comando **genrsa** de OpenSSL para generar un par de claves pública/privada de 1024 bits. Esto es posible porque el algoritmo RSA es asimétrico. También usa **aes128**, un algoritmo de clave simétrica, para encriptar la clave privada que Alice genera usando **genrsa**.

Después de ingresar el comando, OpenSSL le pide a Alice una frase de contraseña, que debe ingresar cada vez que quiera usar las claves:

```
[analyst@secOps lab.support.files]$ openssl genrsa -aes128 -out alice_private.pem 1024
Generating RSA private key, 1024 bit long modulus (2 primes)
.........+++++
.....................................+++++
e is 65537 (0x010001)
Enter pass phrase for alice_private.pem:
Verifying - Enter pass phrase for alice_private.pem:
```

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco


```
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ ls -l alice_private.pem
-rw-------. 1 alice alice 966 Mar 22 17:44 alice_private.pem
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ file alice_private.pem
alice_private.pem: PEM RSA private key
[analyst@secOps lab.support.files]$
```

Bob sigue el mismo procedimiento para generar su par de claves.

Si tiene curiosidad sobre cómo se ve el archivo de clave, puede abrir el archivo **.pem** que generó, pero todo lo que verá es un montón de texto en la pantalla:

```
[analyst@secOps lab.support.files]$ head alice_private.pem
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: AES-128-CBC,E26FAC1F143A30632203F09C259200B9

pdKj8Gm5eeAOF0RHzBx8l1tjmA1HSSvy0RF42bOeb7sEVZtJ6pMnrJ26ouwTQnkL
JJjUVPPHoKZ7j4QpwzbPGrz/hVeMXVT/y33ZEEA+3nrobwisLKz+Q+C9TVJU3m7M
/veiBO9xHMGV01YBNeic7MqXBkhIrNZW6pPRfrbjsBMBGSsL8nwJbb3wvHhzPkeM
e+wtt9S5PWhcnGMj3T+2mtFfW6HWpd8Kdp60z7Nh5mhA9+5aDWREfJhJYzl1zfcv
BmxjJf2wZ3sFJNty+sQVajYfk6UXMyJIuWgAjnqjw6c3vxQi0KE3NUNZYO93GQgEF
pyAnN9uGUTBCDYeTwdw8TEzkyaL08FkzLfFbS2N9BDksA3rpI1cxpxRVFr9+jDBz
[analyst@secOps lab.support.files]$
```

Para ver los detalles de la clave, puede usar el siguiente comando de OpenSSL ingresando el archivo **.pem** y mostrar el contenido, tenga en cuenta que se le pedirá la frase contraseña.

[analyst@secOps lab.support.files]$ **openssl rsa -in alice_private.pem -noout -text**

Es posible que se pregunte dónde encontrar la otra clave, ya que se trata de un solo archivo. Ésta es una buena observación. A continuación, le indicamos cómo obtener la clave pública:

a) Extraer la clave pública

Recuerde, la clave pública es la que puede compartir libremente con otros, mientras que debe mantener su clave privada en secreto. Entonces, **Alice** debe extraer su clave pública y guardarla en un archivo usando el siguiente comando:

[analyst@secOps lab.support.files]$ **openssl rsa -in alice_private.pem -pubout > alice_public.pem**
Enter pass phrase for alice_private.pem:
writing RSA key
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ **ls -l *.pem**

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

-rw-------. 1 alice alice 966 Mar 22 17:44 alice_private.pem
-rw-rw-r--. 1 alice alice 272 Mar 22 17:47 alice_public.pem
[analyst@secOps lab.support.files]$

Puede ver los detalles de la clave pública de la misma manera que antes, pero esta vez, ingrese el archivo **alice_public.pem** de clave pública en su lugar:

[analyst@secOps lab.support.files]$ **openssl rsa -in alice_public.pem -pubin -text -noout**

b) Intercambiar la clave pública

El siguiente paso es que tanto **Alice** como **Bob** deben encontrar el modo de intercambiar sus claves públicas. Por ejemplo adjuntándolas en un correo electrónico o mensaje utilizando redes sociales.

c) Intercambiar mensajes encriptados con la clave públicas

Ahora si, digamos que **Alice** necesita comunicarse en secreto con **Bob**. Escribe su mensaje secreto en un archivo y lo guarda en **top_secret.txt**. Como se trata de un archivo normal, cualquiera puede abrirlo y ver su contenido. No hay mucha protección aquí:

[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ **echo "vim or emacs ?" > top_secret.txt**
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ **cat top_secret.txt**
vim or emacs ?

[analyst@secOps lab.support.files]$

Para cifrar este mensaje secreto, **Alice** necesita utilizar el comando **openssls -encrypt**. Para ello, Necesita proporcionar estas tres cosas:

✔ El nombre del archivo que contiene el mensaje secreto.
✔ Clave pública de **Bob** (archivo)
✔ El nombre de un archivo donde se almacenará el mensaje cifrado

[analyst@secOps lab.support.files]$ **openssl rsautl -encrypt -inkey bob_public.pem -pubin -in top_secret.txt -out top_secret.enc**
[analyst@secOps lab.support.files]$
[analyst@secOps lab.support.files]$ **ls -l top_secret.***
-rw-rw-r--. 1 alice alice 128 Mar 22 17:54 top_secret.enc
-rw-rw-r--. 1 alice alice 15 Mar 22 17:53 top_secret.txt
[analyst@secOps lab.support.files]$

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

Después del cifrado, el archivo original todavía se puede ver, mientras que el archivo cifrado recién creado aparece como un galimatías en la pantalla. Puede estar seguro de que el mensaje secreto se ha cifrado. Probar ejecutar los siguientes comandos:

[analyst@secOps lab.support.files]$ **hexdump -C ./top_secret.enc**

[analyst@secOps lab.support.files]$ **file top_secret.enc**

Es una medida de seguridad eliminar el archivo en texto plano con el mensaje secreto para eliminar cualquier rastro del mismo.

Ahora **Alice** necesita enviar este archivo cifrado a **Bob** a través de la red. Recuerde, incluso si el archivo es interceptado, su contenido está encriptado, por lo que el contenido no se puede revelar.

Parte 2. Descifrar mensajes con OpenSSL utilizando criptografía asimétrica

a) Desencriptar mensaje cifrado utilizando la clave privada

Bob necesita hacer su parte descifrando el mensaje usando **openssl**, pero esta vez usando como argumento de línea de comandos **-decrypt**. Patra lograrlo necesita proporcionar la siguiente información:

✔ El archivo encriptado (que obtuvo de **Alice**)

✔ La propia clave privada de **Bob** (para descifrar, ya que se cifró con la clave pública de **Bob**) ✔

Un nombre de archivo para guardar la salida descifrada a través de la redirección

[analyst@secOps lab.support.files]$ **openssl rsautl -decrypt -inkey bob_private.pem -in top_secret.enc > top_secret.txt**

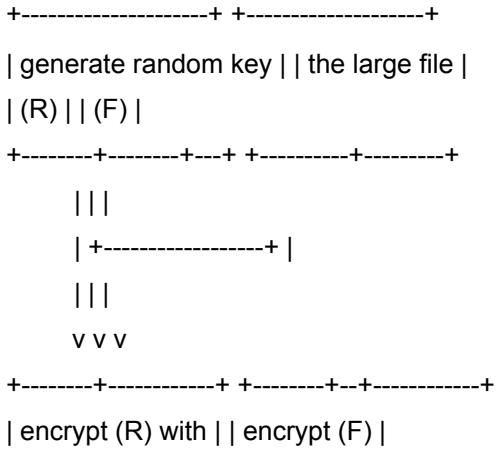
b) Bob puede repetir el proceso

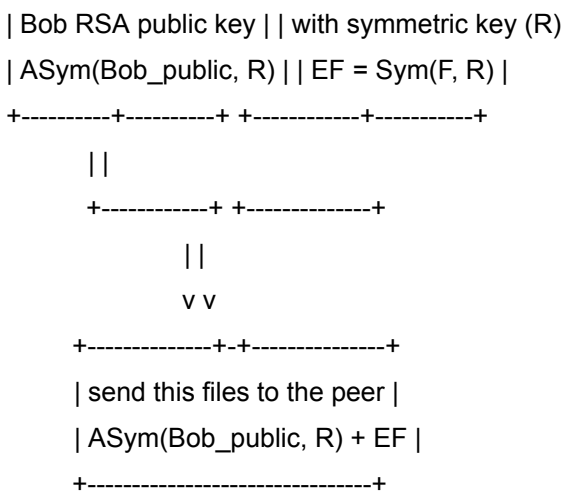
Para enviar su mensaje, **Bob** sigue el mismo proceso que usó **Alice**, pero como el mensaje está destinado a **Alice**, usa la clave pública de **Alice** para cifrar el archivo.

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

Práctica de laboratorio: Cifrar y descifrar grandes archivos con OpenSSL

Ahora bien, cuando hay que cifrar grandes volúmenes de información, la criptografía de clave pública y el algoritmo RSA, no son la mejor alternativa. Para ello, se propone la siguiente solución. Por un lado tenemos el archivo enorme **(F)** y la clave pública de un destinatario **(Bob_public.pem)**. Primero generamos una clave **(R)** al azar utilizando **openssl**. Segundo encriptamos en archivo **(F)** con **(R)** generando **EF = Sym(F, R)**. Tercero encriptamos la clave **(R)** con **Bob_public.pem** utilizando RSA, generando **Asym(Bob_public, R)**. Por último enviamos a **Bob EF** y **Asym(Bob_public, R)**. **Bob** podrá recuperar la clave **(R)** con su clave privada y luego podrá desencriptar **EF**. A continuación se describen los flujos para ambos pasos, encripción y desencripción.





a) **Generate random key**

[analyst@secOps lab.support.files]$ **openssl rand -base64 32 > key.bin**

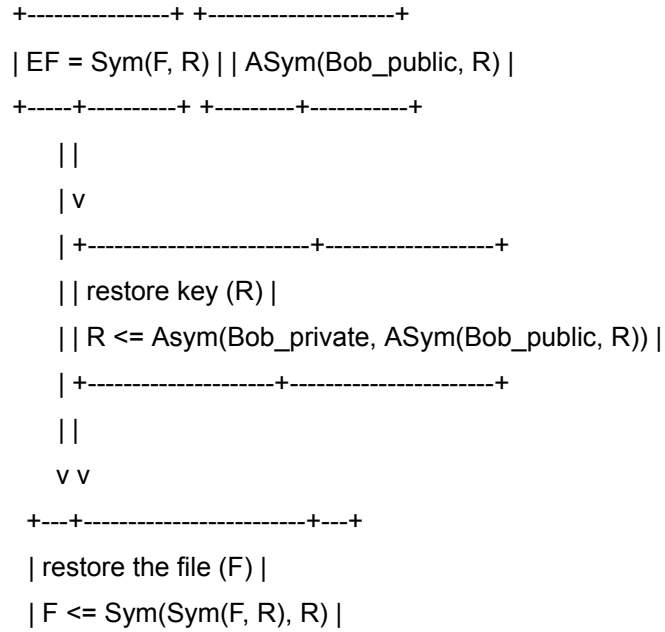
b) **Encrypt (R) with Bob RSA public key**

[analyst@secOps lab.support.files]$ **openssl rsautl -encrypt -pubin -inkey Bob_public.pem -in key.bin -out key.bin.enc**

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco

c) **Encrypt (F) with simmetric key (R)**

[analyst@secOps lab.support.files]$ **openssl aes-256-cbc -a -pbkdf2 -salt -in file_secret -out file_secret.enc -k key.bin**



+--------------------------------+

d) **Restore key (R)**

[analyst@secOps lab.support.files]$ **openssl rsautl -decrypt -inkey Bob_private.pem -in key.bin.enc -out key.bin**

e) **Restore the file (F)**

[analyst@secOps lab.support.files]$ **openssl aes-256-cbc -d -a -in file_secret.enc -out file_secret -k key.bin**

© Cisco y/o sus filiales. Todos los derechos reservados. Información confidencial de Cisco